

SLM Control Software — Progress Report

Project: C:\Users\Nanyte\Desktop\OAM phase\ — a Python SLM (spatial light modulator) control application, grown from the original Mathematica vortex/fork-hologram prototype. Report date: 2026-07-16 Hardware status this session: none connected — everything below is validated by mocks, synthetic ground-truth data, or physics simulation, NOT on a real SLM/camera/instrument.

1. Summary

This session took the project from a Mathematica prototype to a working, GUI-driven Python SLM control application with an automated-measurement scripting API, a full instrument-driver layer, and camera-feedback self-calibration — all developed and tested without hardware. The engine (`slm_toolbox/`) stays importable with only numpy/scipy/pillow+pyserial; every hardware/vendor dependency is optional and lazily imported.

The single durable design decision throughout: additive-overlay phase composition (generate each phase term, sum, wrap to $[0, 2\pi)$, apply a per-wavelength gray $\leftrightarrow$ phase calibration LUT, render, project) — matching both the Mathematica prototype and surveyed vendor (HOLOEYE) software.

2. What exists now — module map

<code>slm_toolbox/</code>	
<code>grid.py</code>	pixel + physical (meter) coordinate grids; every generator takes center=
<code>patterns.py</code>	vortex, blazed grating, beam-position (steer-to-spot), Fresnel lens, axicon, LG mode, Zernike, checkerboard, random
<code>compose.py</code>	additive sum + wrap to $[0, 2\pi)$
<code>render.py</code>	wrapped phase $\rightarrow$ grayscale frame (optional calibration LUT); PNG/BMP save
<code>calibration.py</code>	per-wavelength gray $\leftrightarrow$ phase curve, library, $1/\lambda$ rescale, efficiency inversion
<code>calibration_patterns.py</code>	raw gray-level test patterns for measuring a curve (Method A/C)
<code>display.py</code>	HDMI monitor enumeration + borderless native-res projector window
<code>simulate.py</code>	FFT far-field (Fraunhofer) beam-shape prediction + physics metrics
<code>feedback.py</code>	camera-frame image metrics (sharpness, find_spot, auto_roi, saturation)
<code>autocalibrate.py</code>	SPGD closed-loop self-calibration (wavefront/aberration correction)
<code>api.py</code>	SLM class – scripting entry point for automated measurements
<code>instruments/</code>	
<code>base.py</code>	Instrument base (context manager) + lazy_import helper
<code>elliptec.py</code>	Thorlabs Elliptec: ELLB/ELLC bus + ELL14 rotation / ELL15 / generic device
<code>power_meter.py</code>	Thorlabs PM100 (pyvisa/SCPI; also PM100D/USB/PM160/PM400)
<code>camera.py</code>	HIKROBOT CMOS camera (MVS SDK)
<code>mock.py</code>	hardware-free mocks of every driver + SimFeedbackCamera
<code>phase_gui.py</code>	Tkinter GUI (layered composer + projection + calibration + camera panel)
<code>run_gui.bat</code>	Windows launcher (self-installs deps)
<code>examples/</code>	runnable end-to-end demos (all standalone, no hardware)
<code>tests/</code>	head-less regression scripts (test_simulate/instruments/autocalibrate)
<code>calibration_SOP.md/.pdf</code>	how to measure a real gray $\rightarrow$ phase curve per wavelength
<code>instruments_integration_prompt.md</code>	ready-to-run prompt for the deferred instruments-codes work

3. Capabilities delivered this session

Phase generation

- Full catalog of phase structures beyond the original vortex+grating: focusing (Fresnel lens), axicon (Bessel/ring beams), Zernike aberration terms, LG modes, checkerboard/random test patterns.
- Fork-hologram output verified to match the original Mathematica grating `L=<n>_.jpg` fringe-for-fringe.
- Center offset: global (shifts every layer) + per-layer, exposing the center= kwarg the engine always had.
- Beam steering / spot positioning: `beam_position_phase / SLM.steer_to(dx, dy, f)` — parameterize by target spot position instead of grating period; validated against the far-field simulator (focus lands at the predicted position).

GUI (`phase_gui.py`)

- Layered pattern composer (add/edit/reorder/toggle layers, per-layer weight + center).
- Live preview + full-resolution scrollable view.
- Projection panel: send the pattern to the SLM's HDMI monitor at native resolution (no resampling), auto-selecting the secondary display.
- Calibration panel: enter/load a curve, Method C wizard, Method A pattern.
- Camera + self-calibration panel (new): connect a (simulated or HIKROBOT) camera, grab frames, one-click Self-Calibrate Aberration.

Wavelength-dependent calibration

- `gray<->phase` is nonlinear AND wavelength-dependent ($\Delta\phi$ scales as $1/\lambda$). `calibration.py` stores a measured curve per wavelength, inverts it to the render LUT, and auto-applies it by the Wavelength field.
- Ships with zero fabricated calibration numbers — falls back to a clearly-labeled uncalibrated linear map until a real curve is measured.
- `calibration_SOP.md` (+ PDF): physics + three measurement methods (self-referenced interferometry, Twyman-Green, diffraction-efficiency), sourced from HOLOEYE docs and two peer-reviewed papers.
- Method C fully automated (`run_efficiency_calibration(measure_fn)` / GUI wizard).

Automated-measurement scripting API (`api.py`)

- SLM context-manager class: pattern generators pre-bound to shape/wavelength, calibrated display, fast-sweep window reuse, and `run_efficiency_calibration(measure_fn)` that drives any instrument read callback through a full calibration with no manual data entry.

Far-field simulation (`simulate.py`)

- FFT Fraunhofer prediction of the actual beam a hologram produces (doughnut, ring, focused spot) — no hardware. Doubles as a physics regression test.

Instrument drivers (`instruments/`)

- Thorlabs Elliptec (ELL14 rotation, ELL15, ELLB/ELLC bus), PM100 power meter, HIKROBOT camera — built after reading the user's existing `instruments` codes/ drivers so they reuse the proven library choices (`thorlabs_elliptec`, `VISA/get_power()`).
- Detector drivers expose `measure_fn()` -> `(( ) -> float)` that plugs straight into the calibration / sweep API.

Camera-feedback self-calibration (`feedback.py` + `autocalibrate.py`)

- Sensorless adaptive optics: image the beam -> sharpness metric -> SPGD finds the Zernike correction that cancels the system aberration, per wavelength. OpenCV optional (numpy/scipy do the image math).

4. Verification status

Capability	How verified	On real hardware?
Phase generators / fork holograms	Visual parity vs. Mathematica reference	n/a (pure compute)
Far-field simulator	Physics regression test (exact vortex null, ring growth)	n/a
Center offset / beam steering	Numeric cross-check + far-field simulation	No
Projection (HDMI)	Win32 window introspection (correct class/size/position)	Partial (not eyeballed on a real SLM)
Wavelength calibration	Synthetic ground-truth recovery	No (no measured data)
Instrument drivers	Mocks + clear-error checks; APIs matched to existing working code	No
Camera self-calibration	SimFeedbackCamera loop recovers injected aberration near-exactly (3–4.5× sharpness)	No

Head-less regression suites pass: `tests/test_simulate.py`, `tests/test_instruments.py`, `tests/test_autocalibrate.py`.

5. Key technical findings

- Square-grid vortex null: a vortex beam's on-axis far-field null is exact only when the topological charge 1 is not a multiple of 4 — the pixel grid's 4 -fold symmetry leaves a real residual otherwise. Applies to any pixelated SLM, not just the simulation. Avoid 1 divisible by 4 if exact-center darkness matters.
- Normalized SPGD is essential: the raw SPGD update under-converged badly because the sharpness metric's absolute values are ~ 0.03 ; the scale-invariant normalized update (proportional to $(J+J-)/(|J+|+|J-|)$) fixed convergence.
- Self-calibration base matters: aberration correction converges cleanly on a flat/focus base (near-exact recovery at ~ 100 iters) but only partially on a vortex+grating fork — calibrate system aberration on a simple focus.
- Existing `instruments` codes/ uses the real HOLOEYE HEDS SDK (`import HEDS, HEDS.SLM.Init, showVortex`) — i.e. real vendor-SDK hardware access exists, a materially different path from this project's generic HDMI-monitor approach. Deliberately left un-integrated (see below).
- Camera-read Method C needs the ROI on the $+1$ diffraction order (not the 0 order); it can't be simulated through the aggressive downsample (fine gratings alias) — real-hardware only.

6. Known limitations / not done

- No real-hardware validation of anything. No SLM, camera, or instrument was connected — all green checks are mocks / synthetic / simulation. The HIKROBOT camera driver especially was written to the MVS sample flow but never run against a device (verify pixel-format/stride first).
- No measured calibration data — `calibrations/` is empty until the SOP is run on real hardware.
- HOLOEYE HEDS-SDK SLM integration deferred — the generic HDMI-monitor path is what's built; the vendor-SDK path (which the user's existing code uses) is captured in `instruments_integration_prompt.md` for a dedicated pass.
- Other lab instruments not wired (Andor spectrograph, NI-DAQ, Keithley 4200, lock-in).
- ELL15 semantics unverified — not a datasheet-confirmed model; motion units to confirm on first connection.

- Not built: Airy-beam / vector-vortex generators; persisting the last-used monitor selection; running self-calibration in a worker thread (currently blocks GUI interaction during the loop).
-

7. Recommended next steps

- 1 First hardware bring-up (when an SLM + camera are available): confirm the HDMI projection visually, run the Method C calibration wizard to produce a real `calibrations/*.json`, and bench-test the HIKROBOT driver.
- 2 Decide the SLM control path: generic HDMI monitor (built) vs. the HOLOEYE HEDS SDK (the user's existing code) — resolve before deeper hardware work; prompt is ready in `instruments_integration_prompt.md`.
- 3 Validate the self-calibration loop on the real optics (real camera + real aberrations).
- 4 Lower priority: worker-thread self-calibration with a Stop button; Airy/vector-vortex generators; wire the remaining instruments.